PROJECT CODE

MAIN

```
#include "header.h"
volatile u8 sample_buf[6];

u32 ir_buffer[BUFFER_SIZE];
u32 red_buffer[BUFFER_SIZE];

int main(void)
{
    char buf[30];
    int bpm, spo2;
    float temperature;
    int abnormal;
    unsigned int temp_value;
    u8 read_value;

    IODIR0 |= WARNING_LED;

    uart0_init(9600);
    adc_init();
    i2c_init();
    MAX_init();

    while(1)
    {
        abnormal = 0;
        uart0_tx_string("\r\nPATIENT MONITORING STARTED\r\n");
        uart0_tx_string("--------------------------\r\n");
```

```
/* Read temperature from LM35 */
temperature = LM35_temp();
sprintf(buf, "Temperature = %.2f C\r\n", temperature);
uart0_tx_string(buf);

/* Read MAX30102 sensor samples */
collect_samples();

/* Calculate heart rate and SpO2 */
bpm = calculate_hr();
spo2 = calculate_spo2();

uart0_tx_string("Heart Rate = ");
if(bpm > 0)
{
   uart0_integer(bpm);
   uart0_tx_string(" BPM\r\n");
}
else
{
   uart0_tx_string("Invalid\r\n");
}

uart0_tx_string("SpO2 = ");
if(spo2 >= 0 && spo2 <= 100)
{
   uart0_integer(spo2);
   uart0_tx_string(" %\r\n");
}
else
```

```
{
   uart0_tx_string("Invalid\r\n");
}

/* Check patient status */
if(temperature >= TEMP_HIGH)
   abnormal = 1;
if(bpm <= 0 || bpm < HR_LOW || bpm > HR_HIGH)
   abnormal = 1;
if(spo2 < SPO2_LOW || spo2 > 100)
   abnormal = 1;

if(abnormal)
{
   uart0_tx_string("ABNORMAL CONDITION\r\n");
   IOCLR0 = WARNING_LED;
}
else
{
   uart0_tx_string("PATIENT STATUS: NORMAL\r\n");
   IOSET0 = WARNING_LED;
}

/* Store temperature in addresses 0x00 and 0x01 */
temp_value = (unsigned int)(temperature * 100);
i2c_write(0xA0, 0x00, temp_value & 0xFF);
delay_ms(10);

i2c_write(0xA0, 0x01, (temp_value >> 8) & 0xFF);
delay_ms(10);
```

```
      /* Store heart rate at address 0x02 */
      if(bpm >= 0 && bpm <= 255)
      {
         i2c_write(0xA0, 0x02, bpm);
         delay_ms(10);
      }

      /* Store SpO2 at address 0x03 */
      if(spo2 >= 0 && spo2 <= 100)
      {
         i2c_write(0xA0, 0x03, spo2);
         delay_ms(10);
      }

      uart0_tx_string("EEPROM STORAGE COMPLETED\r\n");
      uart0_tx_string("--------------------------\r\n");

      delay_ms(3000);
   }
}
```

MAX30102 Sensor Data Acquisition and Heart Rate & SpO_2 Calculation Algorithm

```
#include"header.h"
extern volatile u8 sample_buf[6];
   u32 red_data[300];
u32 ir_data[300];
u32 red_raw,IR_raw;
void MAX_init(void)
{
i2c_send(0xAE,0x09,0x40);
while(i2c_receive(0xAE,0X09)&0X40);
i2c_send(0xAE,0x08,0x00);
i2c_send(0xAE,0x0A,0x27);
i2c_send(0xAE,0x0C,0x3F);
i2c_send(0xAE,0x0D,0x24);
i2c_send(0xAE,0x09,0x03);
i2c_send(0xAE,0x04,0x00);
i2c_send(0xAE,0x05,0x00);
i2c_send(0xAE,0x06,0x00);
}

void max30102_read(u32 *red,u32 *ir)
{
i2c_burst_read(0xAE,0x07);
*red= (((u32)sample_buf[0] & 0x03 )<<16) | ((u32)sample_buf[1]<<8) | sample_buf[2];
*red &= 0x03ffff;

*ir= (((u32)sample_buf[3] & 0x03) <<16) | ((u32)sample_buf[4]<<8) | sample_buf[5];
*ir &= 0x03ffff;
}
```

//Collect samples

```
void collect_samples(void)
{
int i;
for(i=0;i<300;i++)
{
max30102_read(&red_data[i],&ir_data[i]);
}
}
```

//to calculate HR(HEART RATE)

```
int calculate_hr(void)
{
u32 min = 0x3ffff,max=0;
u32 threshold;
int i,last_peak=-1;
int intervals=0;
int total_intervals=0;

for(i=0;i<300;i++)
{
if(ir_data[i] < min)
min = ir_data[i];

if(ir_data[i] > max)
max=ir_data[i];
}

if(max <= min)
return 0;
```

```
threshold = min+(max-min)/2;


for(i=1;i<300-1;i++)
{
  if(ir_data[i] > threshold && ir_data[i] >= ir_data[i-1] && ir_data[i] > ir_data[i+1])
   {
     if(last_peak<0)
   {
      last_peak=i;
   }
    else if(i-last_peak >= 30)
   {
   total_intervals +=i-last_peak;
   intervals++;
   last_peak=i;
   }
 }
}
if(intervals == 0)
return 0;
//100 samples per second
return (6000 * intervals) / total_intervals;
}
```

//calculate spo2

```
int calculate_spo2(void)
{
u32 red_min=0x3ffff,red_max=0;
u32 ir_min=0x3ffff,ir_max=0;
```

```
u64 red_sum=0,ir_sum=0;
u32 red_ac,ir_ac,red_dc,ir_dc;
float ratio,spo2;
int i;

for(i=0;i<300;i++)
{
if(red_data[i] < red_min)
red_min=red_data[i];

if(red_data[i] > red_max)
red_max=red_data[i];

if(ir_data[i] < ir_min)
ir_min=ir_data[i];

if(ir_data[i] > ir_max)
ir_max=ir_data[i];

red_sum += red_data[i];
ir_sum += ir_data[i];
}
red_ac=red_max-red_min;
ir_ac=ir_max-ir_min;

red_dc=red_sum/300;
ir_dc=ir_sum/300;

if(red_dc==0 || ir_dc==0 || red_ac==0 || ir_ac==0)
return -1;
```

```
ratio=((u64)red_ac * ir_dc * 1000) / ((u64)red_dc * ir_ac);

spo2 = 104 - (17*ratio)/1000;

if(spo2<0 || spo2 > 100)
return -1;

return spo2;
}
```

DRIVERS – UART DRIVER

```
#include"header.h"
void uart0_init(u32 baud)
{
int pclk,result=0;
if(VPBDIV==0x0)
pclk=15000000;
else if(VPBDIV==0x01)
pclk=6000000;
else if(VPBDIV==0x02)
pclk=30000000;
result=pclk/(16*baud);
PINSEL0 = 0x00000005;
U0LCR=0x83;
U0DLL=result&0xff;
U0DLM=(result>>8)&0xff;
U0LCR=0x03;
}
#define THRE ((U0LSR>>5)&1)
void uart0_tx(u8 data)
{
U0THR=data;
while(THRE==0);
}
#define RDR (U0LSR&1)
u8 uart0_rx(void)
{
while(RDR==0);
return U0RBR;
}
```

```
void uart0_tx_string(char *ptr)
{
while(*ptr!=0)
{
uart0_tx(*ptr);
ptr++;
}
}
uart0_rx_string(u8 *ptr,int len)
{
int i;
for(i=0;i<len;i++)
{
while(RDR==0);
ptr[i]=U0RBR;
if(ptr[i]=='\r')

break;
}
ptr[i]='\0';   }

void uart0_integer(u32 num)
{
int a[10],i=0;
if(num==0)
{
uart0_tx('0');
return;
}
while(num>0)
```

```
{
a[i]=(num%10)+48;
num=num/10;
i++;
}
for(i=i-1;i>=0;i--)
{
uart0_tx(a[i]);
}
}

void uart0_float(float f)
{
int num;
if(f<0)
{
f=-f;
uart0_tx('-');
}
if(f==0)
{
uart0_tx_string("0.0");
return;
}
num=f;
uart0_integer(num);
uart0_tx('.');
num=(f-num)*1000000;
uart0_integer(num);
}
```

I2C DRIVER

```
#include"header.h"
#define SI ((I2CONSET >> 3)&1)
void i2c_init(void)
{
// BIT FREQ SETTING
//Bit frequency = fclk / (I2SCLL + I2SCLH)
PINSEL0 |= 0x50;      //p0.2->SCl p0.3->SDA
I2SCLL = 75;
I2SCLH = 75;

I2CONSET = (1<<6);
I2CONCLR= (1<<2);
}

void i2c_send(u32 slave_addr,u32 memory_addr,u8 data)
{
//step 1: Generate start condition
I2CONSET = (1<<5);   //STA = 1;
I2CONCLR = (1<<3);   //CLEAR SI
while(SI==0);                  //waiting for start conditon to generate
if(I2STAT  != 0x08)            //Checking satart conditon
{
uart0_tx_string("\r\n ERROR : START Conditon\r\n");
goto exit;
}
I2CONCLR = (1<<5);  //STA = 0

//step 2 : send SA+W & check ACK
I2DAT = slave_addr;            //send SA+W
```

```
I2CONCLR = (1<<3);  //SI = 0;
while(SI==0);
if(I2STAT != 0x18)          //checking SA+W & ACK
{
uart0_tx_string("\r\n ERROR : SA+W & ACK failed \r\n");
goto exit;
}
if(I2STAT == 0x20)          //checking SA+W & NO ACk
{
uart0_tx_string("\r\n ERROR : SA+w failed , NO ACK \r\n");
goto exit;
}
```

//step 3 send memory addr & check ACK

```
I2DAT = memory_addr;                //send memory addres
I2CONCLR = (1<<3);     //Clear SI
while(SI==0);
if(I2STAT != 0x28)
{
uart0_tx_string("\r\n ERROR : memory_addr & ACK \r\n");
goto exit;
}
if(I2STAT == 0x30)
{
uart0_tx_string("\r\n ERROR : memory_addr  failed , NOT ACK \r\n");
goto exit;
}
```

//step 4 send data & Check ACK

```
I2DAT = data;
I2CONCLR = (1<<3);            // clear SI
```

```
while(SI==0);
if(I2STAT  != 0x28)
{
uart0_tx_string("\r\n ERROR : Data  & ACK \r\n");
goto exit;
}
if(I2STAT == 0x30)
{
uart0_tx_string("\r\n ERROR : Data  sent , NOT ACK \r\n");
goto exit;
}
//step 5 Generate STOP CONDITION
exit:
I2CONSET = (1<<4); //STO = 1
I2CONCLR = (1<<3);
}

u8 i2c_receive(u32 slave_addr,u32 memory_addr)
{
u8 result;
//STEP 1: Generate start condition
I2CONSET = (1<<5);
I2CONCLR = (1<<3);
while(SI==0);
I2CONCLR=(1<<5);

//STEP 2: Send SA+w & check ACK;
I2DAT = slave_addr;  //send SA+W
I2CONCLR = (1<<3);  //SI = 0;
while(SI==0);
```

```
if(I2STAT != 0x18)                  //checking SA+W & ACK
{
uart0_tx_string("\r\n ERROR : SA+W & ACK failed \r\n");
goto exit;
}
if(I2STAT == 0x20)              //checking SA+W & NO ACk
{
uart0_tx_string("\r\n ERROR : SA+w failed , NO ACK \r\n");
goto exit;
}
```

//step 3: Send memory addr & check ACK

```
I2DAT = memory_addr;   //send memory addres
I2CONCLR = (1<<3);     //Clear SI
while(SI==0);
if(I2STAT != 0x28)
{
uart0_tx_string("\r\n ERROR : memory_addr & ACK \r\n");
goto exit;
}
if(I2STAT == 0x30)
{
uart0_tx_string("\r\n ERROR : memory_addr  failed , NOT ACK \r\n");
goto exit;
}
```

//step 4: Generate restart condition

```
I2CONSET = (1<<5);
I2CONCLR = (1<<3);
while(SI==0);
if(I2STAT != 0x10)
```

```
{
uart0_tx_string("\r\n ERROR : Restart conditon\r\n");
goto exit;
}
I2CONCLR = (1<<5);
//STEP 5 : send SA+R & check ack
I2DAT = slave_addr|1;
I2CONCLR = (1<<3);
while(SI==0);
if(I2STAT != 0x40)
{
uart0_tx_string("\r\n ERROR : SA+R sent, ACK\r\n");
goto exit;
}
if(I2STAT == 0x48)
{
uart0_tx_string("\r\n ERROR : SA+R sent, No ACK\r\n");
goto exit;
}
// step 6: read data from slave& send no ack
//I2CONCLR = (1<<2);
I2CONCLR = (1<<3);
while(SI==0);
if(I2STAT != 0x58)
{
uart0_tx_string("\r\n ERROR : DAta Reception\r\n");
goto exit;
}
result =I2DAT;
```

```
// step 7:generate stop condition
exit:
I2CONSET = (1<<4);
I2CONCLR = (1<<3);

return result;
}

//I2C burst read to read multiple bytes of sensor data continously in single i2c

void i2c_burst_read(u32 slave_addr,u32 memory_addr)
{
s8 i;
//Generate start condition
I2CONSET = (1<<5);
I2CONCLR = (1<<3);
while(SI == 0);
if(I2STAT != 0x08)
{
uart0_tx_string("\r\n ERROR : Start Condition\r\n");
goto exit;
}
I2CONCLR = (1<<5);

//sending SA+W
I2DAT = slave_addr;
I2CONCLR = (1<<3);
while(SI==0);
if(I2STAT != 0x18)
{
```

```
 uart0_tx_string("\r\n ERROR : SA+W failed \r\n");
goto exit;
}
//sending memory addr
I2DAT= memory_addr;
I2CONCLR = (1<<3);
while(SI==0);

if(I2STAT != 0x28)
{
uart0_tx_string("\r\n ERROR : Memory addr, ACK \r\n");
goto exit;
}
//send restart conditon again
I2CONSET = (1<<5);
I2CONCLR = (1<<3);
while(SI==0);
if(I2STAT != 0x10)
{
uart0_tx_string("\r\n ERROR : Restart conditon\r\n");
goto exit;
}
I2CONCLR = (1<<5);
//STEP 5 : send SA+R & check ack
I2DAT = slave_addr | 1;
I2CONCLR = (1<<3);
while(SI==0);
if(I2STAT != 0x40)
{
uart0_tx_string("\r\n ERROR : SA+R sent & ACK\r\n");
```

```
goto exit;
}
for(i=0;i<6;i++)
{
// step 6 read data from slave& send no ack
if(i<5)
I2CONSET = (1<<2);
else
I2CONCLR = (1<<2);

I2CONCLR = (1<<3);
while(SI==0);
if((i<5) && I2STAT != 0x50)
{
uart0_tx_string("\r\n ERROR : Data byte has been received ,NOT ACK returned \r\n");
goto exit;
}
if((i==5) && I2STAT != 0x58)
{
  uart0_tx_string("\r\n ERROR :Data received,ACK returned\r\n");
  goto exit;
}
sample_buf[i] = I2DAT;
}
// step 7:generate stop condition
exit:
I2CONSET = (1<<4);
I2CONCLR = (1<<3);
while((I2CONSET >>4 ) &1 );
}
```

ADC DRIVER (LM35 Temperature)

```
#include"header.h"
void adc_init(void)
{
PINSEL1 |= 0x15400000;
ADCR=0x00200400;
}
#define DONE ((ADDR>>31)&1)

unsigned int adc_read(int ch_num)
{
unsigned int result=0;
ADCR |= (1<<ch_num);
ADCR |= (1<<24);

while(DONE==0);
ADCR &= ~(1<<24);
ADCR &= ~(1<<ch_num);
result = (ADDR>>6&0x3ff);
return result;
}

float LM35_temp(void)
{
unsigned int adc;
float Vout;
float tempt;

adc=adc_read(1);
```

```
Vout=(adc*3.3)/1023;
tempt=(Vout*100);

return tempt;
}
```

DELAY

```
#include"header.h"
void delay_ms(u32 ms)
{
T0PR=15000-1;
T0PC=0;
T0TC=0;
T0TCR=1;
while(T0TC<ms);
T0TCR=0;
}
void delay_sec (u32 sec)
{
T0PR=15000000-1;
T0PC=0;
T0TC=0;
T0TCR=1;
while(T0TC<sec);
T0TCR=0;
}
```

LCD_4BIT_DRIVER

```
#include <LPC21xx.H>
#include"header.h"
void lcd_data(unsigned char data)
{
```

```
unsigned int temp;
IOCLR1=0xFE<<16;
temp=(data & 0xf0)<<16;
IOSET1=temp;
IOSET1=1<<17;
IOCLR1=1<<18;
IOSET1=1<<19;
delay_ms(2);
IOCLR1=1<<19;
// send lower nibble data//
IOCLR1=0xFE<<16;
IOSET1=(data & 0x0f)<<20;
IOSET1=1<<17;
IOCLR1=1<<18;
IOSET1=1<<19;
delay_ms(2);
IOCLR1=1<<19;
}
void lcd_cmd(unsigned int cmd)
{
unsigned int temp;
IOCLR1=0xFE<<16;
temp=(cmd & 0xf0)<<16;
IOSET1=temp;
IOCLR1=1<<17;
IOCLR1=1<<18;
IOSET1=1<<19;
delay_ms(2);
IOCLR1=1<<19;
// send lower nibble data//
```

```
IOCLR1=0xFE<<16;
IOSET1=(cmd & 0x0f)<<20;
IOCLR1=1<<17;
IOCLR1=1<<18;
IOSET1=1<<19;
delay_ms(2);
IOCLR1=1<<19;
}
void lcd_init(void)
{
IODIR0=0xFE<<16;
IOCLR1=1<<19;
lcd_cmd(0x02);
lcd_cmd(0x28);
lcd_cmd(0x0e);
lcd_cmd(0x01);
}
void lcd_integer(int num)
{
int a[10],i;
if(num==0)
{
lcd_data('0');
}
if(num<0)
{
num=-num;
lcd_data('-');
}
while(num>0)
```

```
{
a[i]=num%10+48;
num=num/10;
i++;
}
for(i=i-1;i>=0;i--)
{
lcd_data(a[i]);
}
}
 void lcd_string(char *ptr)
{
while(*ptr!=0)
{
lcd_data(*ptr);
ptr++;
}
}
void lcd_float(float f)
{
int num;
if(f<0)
{
f=-f;
lcd_data('-');
}
if(f==0)
{
lcd_string("0.0");
return;
```

```
}
num=f;
lcd_integer(num);
lcd_data('.');
num=(f-num)*1000000;
lcd_integer(num);
}
```

HEADER

```
#include <LPC21xx.H>
#include<stdio.h>

typedef unsigned int u32;
typedef unsigned long long u64;
typedef signed int s32;
typedef unsigned short int u16;
typedef unsigned char u8;
typedef signed char s8;

#define BUFFER_SIZE 400
#define TEMP_HIGH 38
#define HR_LOW 60
#define HR_HIGH 100
#define SPO2_LOW 95
#define WARNING_LED (1<<7)
//extern
extern u32 ir_buffer[BUFFER_SIZE];
extern u32 red_buffer[BUFFER_SIZE];
extern volatile u8 sample_buf[6];
//DELAY

extern void delay_sec(u32 sec);
extern void delay_ms(u32 ms);

//LCD
extern void lcd_data(u8 data);
extern  void lcd_cmd(unsigned int cmd);
extern void lcd_init(void);
```

```
extern void lcd_string(char *ptr);
extern void lcd_integer( int num);

//UART
extern void uart0_init(u32 baud);
extern void uart0_tx(u8 data);
extern u8 uart0_rx(void);
extern void uart0_tx_string(char *ptr);
extern void uart0_integer(u32 num);

//ADC
extern void adc_init(void);
extern unsigned int adc_read(int ch_num);
extern float LM35_temp(void);

//I2C
extern void i2c_init(void);
extern void i2c_send(u32 slave_addr,u32 memory_addr,u8 data);
extern u8 i2c_receive(u32 slave_addr,u32 memory_addr);
extern void i2c_burst_read(u32 slave_addr,u32 memory_addr);

//MAX30102
extern void MAX_init(void);
extern void max30102_read(u32 *red,u32 *ir);

extern int calculate_spo2(void);
extern int calculate_hr(void);
extern void collect_samples(void);
```